

大地主题解算报告

一、运行方式

运行环境：VS2022 C#

可运行程序文件：大地主题解算代码\bin\Debug\GaussGeodeticCalculator.exe

执行方式：

1. 选择大地基准；
2. 输入待解算的数据；
3. 点击正/反算按钮，得到解算结果。

二、原理

大地主题解算分为正算（已知起点经纬度 L_1, B_1 、大地方位角 A_1 和大地线长度 S ，求终点经纬度 L_2, B_2 及反方位角 A_2 ）和反算（已知两点经纬度 L_1, B_1 、 L_2, B_2 ，求大地线长度 S 及正、反方位角 A_{12} ， A_{21} ），核心是利用椭球面上大地元素（经纬度、方位角、距离）间的数学关系推导公式计算，以下结合高斯平均引数等方法说明：

1. 高斯平均引数正算原理

已知 L_1, B_1, A_1, S ，目标求 L_2, B_2, A_2 ，关键公式基于椭球大地线微分关系推导，通过泰勒展开等方法整理出含平均引数（如平均纬度 B_m 、平均方位角 A_m ）的公式：

纬度差 $\Delta B'' = (B_2 - B_1)''$ 公式:

$$\Delta B'' = \frac{\rho'' V_m^2}{N_m} p'' S \cdot \cos A_m \left\{ 1 + \frac{S^2}{24 N_m^2} [\sin^2 A_m (3 + 3 \eta_m^2 t_m^2) + 3 \eta_m^2 \cos^2 A_m (-1 + t_m^2 - \eta_m^2 - 4 t_m^2 \eta_m^2)] \right\} + 5 \text{ 次项}$$

经度差 $\Delta L'' = (L_2 - L_1)''$ 公式:

$$\Delta L'' = \frac{\rho''}{N_m} S \cdot \sec B_m \sin A_m \left\{ 1 + \frac{S^2}{24 N_m^2} [\sin^2 A_m \cdot t_m^2 - \cos^2 A_m (1 + \eta_m^2 - 9 t_m^2 \eta_m^2 + \eta_m^4)] \right\} + 5 \text{ 次项}$$

方位角差 $\Delta A'' = (A_{21} - A_{12})''$ 公式:

$$\Delta A'' = \frac{\rho''}{N_m} S \cdot \sin A_m t_m \left\{ 1 + \frac{S^2}{24 N_m^2} [\cos^2 A_m (2 + 7 \eta_m^2 + 9 t_m^2 \eta_m^2 + 5 \eta_m^4) + \sin^2 A_m (2 + t_m^2 + 2 \eta_m^2)] \right\} + 5 \text{ 次项}$$

(式中 ρ'' 为弧度秒值, N_m 为卯酉圈曲率半径, V_m 为辅助函数,

$t_m = \tan B_m$, $\eta_m^2 = e'^2 \cos^2 B_m$, e' 为椭球第二偏心率)

因公式中 B_m ($B_m = \frac{B_1+B_2}{2}$)、 A_m ($A_m = \frac{A_1+A_2}{2}$) 含未知量 B_2 , A_2 , 需迭代计算: 先设初始 $B_m^0 = B_1$ 、 $A_m^0 = A_1$ 代入算 $\Delta B'$, $\Delta L'$, $\Delta A'$ 更新 $B_2^1 = B_1 + \Delta B'$ 、 $A_2^1 = A_1 + \Delta A'$, 再算新 B_m^1, A_m^1 , 重复至 $|\Delta B_{i+1} - \Delta B_i|, |\Delta L_{i+1} - \Delta L_i|, |\Delta A_{i+1} - \Delta A_i| < \text{精度阈值} \varepsilon$ 。

2. 高斯平均引数反算原理

已知 L_1, B_1, L_2, B_2 , 目标求 S, A_{12}, A_{21} , 先算纬差 $\Delta B = B_2 - B_1$ 、经差 $\Delta L = L_2 - L_1$, 通过构建方程推导 $S \cdot \sin A_m$ 、 $S \cdot \cos A_m$ 等关系:

定义辅助量 $U = S \cdot \sin A_m$, $V = S \cdot \cos A_m$, 结合椭球关系有:

$$\begin{cases} U = \frac{N_m \Delta L'' \cos B_m}{\rho''} \left(1 + \frac{\Delta A_0^2}{24 \rho''^2} - \frac{\Delta B_0^2}{24 \rho''^2} \right) \\ V = \frac{M_m \Delta B''}{\rho''} \left(1 + \frac{\Delta L_0^2}{12 \rho''^2} + \frac{\Delta A_0^2}{24 \rho''^2} \right) \end{cases}$$

($\Delta A_0, \Delta B_0, \Delta L_0$ 为简化主项, M_m 为子午圈曲率半径) 由

$\tan A_m = \frac{U}{V}$ 确定 A_m , 再结合精度修正公式 (如考虑 η_m^2 等项的修正)

迭代/直接计算 S :

$$S = \frac{U}{\sin A_m} = \frac{V}{\cos A_m}$$

方位角计算:

$$A_{12} = A_m - \frac{1}{2} \Delta A'', A_{21} = A_m + \frac{1}{2} \Delta A'' \pm 180^\circ$$

($\Delta A''$ 为方位角差, 符号依椭球方向规则调整)

三、代码

```
using System;
using System.Windows.Forms;
namespace GaussGeodeticCalculator
{
    public partial class MainForm : Form
    {
        // 椭球体参数
        private double a = 6378137;           // 椭圆长半轴
        private double b = 6356752.3142;      // 椭圆短半轴
        private double e_2 = 0.00669437999013; // 第一偏心率 e^2
        private double e_dot_2 = 0.00673949674227; // 第二偏心率 e'^2
        private const double rou = 206264.806247096355; // 弧度到秒的转换常数
        public MainForm()
        {
            InitializeComponent();
        }
    }
}
```

```

// 预选择 WGS-84 椭球体
radioButtonWGS84.Checked = true;

// 填充算例数据
FillExampleData();
}

private void Ellipsoid_RadioButton_CheckedChanged(object sender, EventArgs e)
{
    RadioButton rb = (RadioButton)sender;
    switch (rb.Text)
    {
        case "克拉索夫斯基椭球":
            a = 6378245;
            b = 6356863.0187730473;
            e_2 = 0.006693421622966;
            e_dot_2 = 0.006738525414683;
            break;
        case "1975 年国际椭球体":
            a = 6378140;
            b = 6356755.2881575287;
            e_2 = 0.006694384999588;
            e_dot_2 = 0.006739501819473;
            break;
        case "WGS-84 椭球体":
            a = 6378137;
            b = 6356752.3142;
            e_2 = 0.00669437999013;
            e_dot_2 = 0.00673949674227;
            break;
        case "2000 中国大地坐标系":
            a = 6378137;
            b = 6356752.3141;
            e_2 = 0.00669438002290;
            e_dot_2 = 0.00673949677548;
            break;
    }
}

private void ForwardCalculation()
{
    try
    {
        // 输入验证
        if (!ValidateInput(tb 正算起点纬度度, tb 正算起点纬度分, tb 正算起点纬度秒,
            tb 正算起点经度度, tb 正算起点经度分, tb 正算起点经度秒,
            tb 正算方位角度, tb 正算方位角分, tb 正算方位角秒,

```

```

        tb 正算大地线长))

    return;
    // 转换输入为弧度
    double B1 = ConvertToRadians(tb 正算起点纬度度.Text, tb 正算起点纬度分.Text, tb 正算起点纬度
秒.Text);
    double L1 = ConvertToRadians(tb 正算起点经度度.Text, tb 正算起点经度分.Text, tb 正算起点经度
秒.Text);
    double A1 = ConvertToRadians(tb 正算方位角度.Text, tb 正算方位角分.Text, tb 正算方位角秒.Text);
    double S = Convert.ToDouble(tb 正算大地线长.Text);
    double c = a * a / b;
    double W = Math.Sqrt(1 - e_2 * Math.Sin(B1) * Math.Sin(B1));
    double V = Math.Sqrt(1 + e_dot_2 * Math.Cos(B1) * Math.Cos(B1));
    double M1 = a * (1 - e_2) / (W * W * W);
    double N1 = a / W;
    double t1 = Math.Tan(B1);
    // 初始迭代值
    double delta_B = V * V / N1 * rou * S * Math.Cos(A1);
    double delta_L = rou / N1 * S / Math.Cos(B1) * Math.Sin(A1);
    double delta_A = rou / N1 * S * Math.Sin(A1) * t1;
    double Bm = B1 + delta_B / rou / 2;
    double Am = A1 + delta_A / rou / 2;
    double tm = Math.Tan(Bm);
    double nm = e_dot_2 * Math.Cos(Bm) * Math.Cos(Bm);
    double Wm = Math.Sqrt(1 - e_2 * Math.Sin(Bm) * Math.Sin(Bm));
    double Vm = Math.Sqrt(1 + e_dot_2 * Math.Cos(Bm) * Math.Cos(Bm));
    double Mm = a * (1 - e_2) / (Wm * Wm * Wm);
    double Nm = a / Wm;
    double chB, chL, chA;
    do
    {
        chB = delta_B;
        chL = delta_L;
        chA = delta_A;
        delta_B = Vm * Vm / Nm * rou * S * Math.Cos(Am) *
            (1 + S * S / 24 / Nm / Nm * (Math.Pow(2 + 3 * tm * tm + 3 * Math.Pow(nm * tm, 2), 2)) + 3 *
            nm * nm * Math.Cos(Am) * (tm * tm - 1 - nm * nm - 4 * tm * tm * nm * nm));
        delta_L = rou / Nm * S / Math.Cos(Bm) * Math.Sin(Am) *
            (1 + S * S / 24 / Nm / Nm * (Math.Sin(Am) * Math.Sin(Am) * tm * tm - Math.Cos(Am) * Math.Co
s(Am) *
            (1 + nm * nm - 9 * tm * tm * nm * nm + Math.Pow(nm, 4))));
        delta_A = rou / Nm * S * Math.Sin(Am) * tm *
            (1 + S * S / 24 / Nm / Nm * (Math.Cos(Am) * Math.Cos(Am) * (2 + 7 * nm * nm + 9 * tm * tm * n
m * nm + 5 * Math.Pow(nm, 4))
            + Math.Sin(Am) * Math.Sin(Am) * (2 + tm * tm + 2 * nm * nm)));
    }

```

```

        Bm = B1 + delta_B / rou / 2;
        Am = A1 + delta_A / rou / 2;
        Wm = Math.Sqrt(1 - e_2 * Math.Sin(Bm) * Math.Sin(Bm));
        Vm = Math.Sqrt(1 + e_dot_2 * Math.Cos(Bm) * Math.Cos(Bm));
        Mm = a * (1 - e_2) / (Wm * Wm * Wm);
        Nm = a / Wm;
        tm = Math.Tan(Bm);
        nm = e_dot_2 * Math.Cos(Bm) * Math.Cos(Bm);
    } while (Math.Abs(chB - delta_B) > 1e-4 || Math.Abs(chA - delta_A) > 1e-
4 || Math.Abs(chL - delta_L) > 1e-4);
    double B2 = (B1 + delta_B / rou) * 180 / Math.PI;
    double L2 = (L1 + delta_L / rou) * 180 / Math.PI;
    double A21 = (A1 + delta_A / rou) * 180 / Math.PI;
    if (A1 > 180)
        A21 = A21 + 180;
    else
        A21 = A21 - 180;
    if (A21 < 0)
        A21 += 360;
    // 转换为度分秒并显示
    ConvertToDMS(B2, tb 正算终点纬度数, tb 正算终点纬度分, tb 正算终点纬度秒);
    ConvertToDMS(L2, tb 正算终点经度数, tb 正算终点经度分, tb 正算终点经度秒);
    ConvertToDMS(A21, tb 正算反方位角度, tb 正算反方位角分, tb 正算反方位角秒);
}
catch (Exception ex)
{
    MessageBox.Show("正算错误: " + ex.Message, "错误
", MessageBoxButtons.OK, MessageBoxIcon.Error);
}
}
private void InverseCalculation()
{
    try
    {
        // 输入验证
        if (!ValidateInput(tb 反算起点纬度数, tb 反算起点纬度分, tb 反算起点纬度秒,
            tb 反算起点经度数, tb 反算起点经度分, tb 反算起点经度秒,
            tb 反算终点纬度数, tb 反算终点纬度分, tb 反算终点纬度秒,
            tb 反算终点经度数, tb 反算终点经度分, tb 反算终点经度秒))
            return;
        // 转换输入为弧度
        double B1 = ConvertToRadians(tb 反算起点纬度数.Text, tb 反算起点纬度分.Text, tb 反算起点纬度
秒.Text);

```

```

double L1 = ConvertToRadians(tb 反算起点经度度.Text, tb 反算起点经度分.Text, tb 反算起点经度
秒.Text);
double B2 = ConvertToRadians(tb 反算终点纬度度.Text, tb 反算终点纬度分.Text, tb 反算终点纬度
秒.Text);
double L2 = ConvertToRadians(tb 反算终点经度度.Text, tb 反算终点经度分.Text, tb 反算终点经度
秒.Text);
double Bm = (B1 + B2) / 2;
double Lm = (L1 + L2) / 2;
double tm = Math.Tan(Bm);
double nm = e_dot_2 * Math.Cos(Bm) * Math.Cos(Bm);
double Wm = Math.Sqrt(1 - e_2 * Math.Sin(Bm) * Math.Sin(Bm));
double Vm = Math.Sqrt(1 + e_dot_2 * Math.Cos(Bm) * Math.Cos(Bm));
double Mm = a * (1 - e_2) / (Wm * Wm * Wm);
double Nm = a / Wm;
double r01 = Nm / rou * Math.Cos(Bm);
double r21 = Nm * Math.Cos(Bm) / 24 / Math.Pow(Vm, 4) / Math.Pow(rou, 3) * (1 + nm * nm - 9 * Mat
h.Pow(nm * tm * nm * nm, 2));
double r03 = -Nm / 24 / Math.Pow(rou, 3) * Math.Pow(Math.Cos(Bm), 3) * tm * tm;
double S10 = Nm / rou / Vm / Vm;
double S12 = Nm * Math.Cos(Bm) * Math.Cos(Bm) / 24 / Math.Pow(Vm, 2) / Math.Pow(rou, 3) * (2 +
3 * tm * tm + 2 * nm * nm);
double S30 = Nm / 8 / Math.Pow(rou, 3) / Math.Pow(Vm, 6) * (nm * nm - tm * tm * nm * nm);
double delta_B = (B2 - B1) * rou;
double delta_L = (L2 - L1) * rou;
double SsinAm = r01 * delta_L + r21 * delta_B * delta_B * delta_L + r03 * Math.Pow(delta_L, 3);
double ScosAm = S10 * delta_B + S12 * delta_B * delta_L * delta_L + S30 * Math.Pow(delta_B, 3);
double t01 = tm * Math.Cos(Bm);
double t21 = Math.Cos(Bm) * tm * (2 + 7 * nm * nm + 9 * tm * tm * nm * nm + 5 * Math.Pow(nm, 4)) /
24 / rou / rou / Math.Pow(Vm, 4);
double t03 = Math.Pow(Math.Cos(Bm), 3) * tm * (2 + tm * tm + 2 * nm * nm) / 24 / rou / rou;
double delta_A = t01 * delta_L + t21 * delta_B * delta_B * delta_L + t03 * Math.Pow(delta_L, 3);
double Am = Math.Atan(SsinAm / ScosAm);
if (Am < 0)
    Am = Am + 2 * Math.PI;
double A1 = Am - delta_A / 2 / rou;
double A21 = Am + delta_A / 2 / rou;
if (A1 > Math.PI)
    A21 = A21 - Math.PI;
if (A1 < Math.PI)
    A21 = A21 + Math.PI;
double S = ScosAm / Math.Cos(Am);
// 转换为度分秒并显示
tb 反算大地线长.Text = S.ToString("F4");
ConvertToDMS(A1 * 180 / Math.PI, tb 反算正方位角度, tb 反算正方位角分, tb 反算正方位角秒);

```

```

        ConvertToDMS(A21 * 180 / Math.PI, tb 反算反方位角度, tb 反算反方位角分, tb 反算反方位角秒);
    }
    catch (Exception ex)
    {
        MessageBox.Show("反算错误: " + ex.Message, "错误", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}

private double ConvertToRadians(string d, string m, string s)
{
    double degree = Convert.ToDouble(d);
    double minute = Convert.ToDouble(m);
    double second = Convert.ToDouble(s);
    return (degree + minute / 60 + second / 3600) * Math.PI / 180;
}

private void ConvertToDMS(double angle, TextBox tbD, TextBox tbM, TextBox tbS)
{
    double d = Math.Floor(angle);
    double m = Math.Floor((angle - d) * 60);
    double s = ((angle - d) * 60 - m) * 60;
    tbD.Text = d.ToString("F0");
    tbM.Text = m.ToString("F0");
    tbS.Text = s.ToString("F4");
}

private bool ValidateInput(params TextBox[] textBoxes)
{
    foreach (TextBox tb in textBoxes)
    {
        if (string.IsNullOrEmpty(tb.Text) || !double.TryParse(tb.Text, out double result))
        {
            MessageBox.Show("请输入有效的数字", "输入错误", MessageBoxButtons.OK, MessageBoxIcon.Warning);
            tb.Focus();
            return false;
        }
    }
    return true;
}

private void FillExampleData()
{
    // 正算算例数据
    tb 正算起点纬度度.Text = "47";
    tb 正算起点纬度分.Text = "46";
    tb 正算起点纬度秒.Text = "52.6470";
}

```



```

        tb 正算起点经度度.Text = "35";
        tb 正算起点经度分.Text = "49";
        tb 正算起点经度秒.Text = "36.3300";
        tb 正算方位角度.Text = "44";
        tb 正算方位角分.Text = "12";
        tb 正算方位角秒.Text = "13.664";
        tb 正算大地线长.Text = "44797.2826";
        // 反算算例数据
        tb 反算起点纬度度.Text = "47";
        tb 反算起点纬度分.Text = "46";
        tb 反算起点纬度秒.Text = "52.6470";
        tb 反算起点经度度.Text = "35";
        tb 反算起点经度分.Text = "49";
        tb 反算起点经度秒.Text = "36.3300";
        tb 反算终点纬度度.Text = "48";
        tb 反算终点纬度分.Text = "04";
        tb 反算终点纬度秒.Text = "09.6384";
        tb 反算终点经度度.Text = "36";
        tb 反算终点经度分.Text = "14";
        tb 反算终点经度秒.Text = "45.0505";
    }

    private void btn 正算_Click(object sender, EventArgs e)
    {
        ForwardCalculation();
    }

    private void btn 反算_Click(object sender, EventArgs e)
    {
        InverseCalculation();
    }

    private void groupBox1_Enter(object sender, EventArgs e)
    {
    }

    private void groupBox3_Enter(object sender, EventArgs e)
    {
    }

    private void MainForm_Load(object sender, EventArgs e)
    {
    }
}

```

四、结果

选择框架、输入起算参数：

大地主题解算 - 张洋20232411

选择框架

☐ 克拉索夫斯基椭球

☐ 1975年国际椭球体

☒ WGS-84椭球体

☐ 2000中国大地坐标系

高斯正算

起点纬度B1: 47 度 46 分 52.6470 秒

终点纬度B2: 度 分 秒

起点经度L1: 35 度 49 分 36.3300 秒

终点经度L2: 度 分 秒

正方位角A12: 44 度 12 分 13.664 秒

反方位角A21: 度 分 秒

大地线长S: 44797.2826 米

正 算

高斯反算

起点纬度B1: 47 度 46 分 52.6470 秒

大地线长S: 米

起点经度L1: 35 度 49 分 36.3300 秒

正方位角A12: 度 分 秒

终点纬度B2: 48 度 04 分 09.6384 秒

反方位角A21: 度 分 秒

终点经度L2: 36 度 14 分 45.0505 秒

反 算

解算示例：

大地主题解算 - 张洋20232411

选择框架

☐ 克拉索夫斯基椭球

☐ 1975年国际椭球体

☒ WGS-84椭球体

☐ 2000中国大地坐标系

高斯正算

起点纬度B1: 47 度 46 分 52.6470 秒

终点纬度B2: 48 度 4 分 9.7234 秒

起点经度L1: 35 度 49 分 36.3300 秒

终点经度L2: 36 度 14 分 45.0760 秒

正方位角A12: 44 度 12 分 13.664 秒

反方位角A21: 224 度 30 分 53.5699 秒

大地线长S: 44797.2826 米

正 算

高斯反算

起点纬度B1: 47 度 46 分 52.6470 秒

大地线长S: 44796.7879 米

起点经度L1: 35 度 49 分 36.3300 秒

正方位角A12: 44 度 12 分 12.5426 秒

终点纬度B2: 48 度 04 分 09.6384 秒

反方位角A21: 224 度 30 分 52.4295 秒

终点经度L2: 36 度 14 分 45.0505 秒

反 算